

Delivery Agent

Artificial Intelligence — Assignment 1

Author: Yaroslav Nesteruk

Table of Contents

1 Project overview	3
Environment	3
Letter	3
2 PEAS	4
3 Environment properties	4
4 Observation	5
5 Evaluation of possible actions	6
Choosing an order and finding a road	6
7 Agent	7
8 Results	7
9 Experiments	7
10 Results and interpretation	8
10 by 10 map	9
20 by 20 map	10
A decision traced through the history and code	10
11 User manual	11
Install the required packages with pip	11
Run a configuration	11
Run the default configuration with the button	11
Find results and change settings	12
Run all tests and show each result	12
12 Libraries and external sources	12

1 Project overview

This program simulates a courier delivering letters on foot. The agent is the part of the program that decides what the courier should do next. The environment is the simulated city: it keeps track of the streets, letters, courier and time. The courier collects letters, carries them in a “bag” and takes each one to its delivery address. The goal is to deliver as many letters as possible before their deadlines.

Environment

The city is a grid of square cells. Each cell has an address written as two coordinates, such as (3, 5). The courier can move one cell up, down, left or right, but not diagonally. A cell represents 500 metres, and a move through the cell takes 6 simulated minutes. Buildings are cells the courier cannot enter. A temporary road closure blocks a street cell for a limited simulated time.

Location	Purpose
Depot	The courier’s starting location, placed at the centre of the map.
Pickup point	A place where letters appear and wait for collection. There can be several, and the courier must visit the point holding a letter to collect it.
Delivery address	The cell where the courier must take a particular letter.

Letter

A letter first appears at a pickup point. The courier receives a notification with its pickup location and delivery address, even if that point is outside its current view. The letter waits there until collected. Once collected, it occupies one space in the “bag” and its delivery timer starts. The courier delivers it only when standing at its address. A late letter can still be delivered.

New letters can appear while the courier is travelling. The “bag” has a limit, set to 10 letters in the current configs.

2 PEAS

PEAS describes four parts of an agent's task: Performance, Environment, Actuators and Sensors.

Part	Meaning in this project
Performance	The share of letters delivered on time
Environment	The grid city, depot, pickup points, letters, buildings, temporary road closures and simulation clock.
Actuators	The mechanisms that carry out the courier's commands: move, collect, deliver or wait.
Sensors	The inputs that tell the agent about visible streets and closures, available letters, position, direction, time, free bag space and the previous action's result.

3 Environment properties

Property	Plain explanation
Partially observable	The courier does not receive all information about the city. It knows the building map, but cannot see future incoming letters or closures outside its view.
Random setup	The program randomly chooses parts of the city and letter events. Movement itself follows fixed rules. A seed is a number used to repeat the random choices with the same configs.
Sequential	Decisions affect later decisions. Travelling to one address changes both the courier's location and the time left for other letters.
Dynamic	Letters and road closures change when actions advance that clock.
Discrete	The model uses separate grid cells, whole minutes and four kinds of action rather than continuous movement.
Single-agent	Only one courier makes decisions. Pickup points and scheduled events do not make decisions of their own.

4 Observation

The class **Environment** stores the actual city, including events the courier cannot yet know. It sends the agent an **AgentObservation**: a package containing the time, courier position and direction, free bag space, depot and pickup locations, letters that have appeared, visible closures and feedback from the previous action. The agent receives descriptions of letters.

The agent receives the building map and movement rules when it is created. During a run, it obtains new letter information and local road observations.

VisionSensor includes the current cell, one straight line forward until a building or map edge, and up to two cells on each side. Buildings stop visibility; the view does not turn corners. Notifications about newly available letters are independent of this local view.

Command	What happens	Time added
MOVE	Move to one neighbouring street cell. Buildings and active closures cannot be entered.	6 min if successful
PICK_UP	Collect one letter at the current pickup point, if the bag has space.	0 min
DELIVER	Drop one carried letter at its delivery address.	0 min
WAIT	Stay in the current cell, let one minute pass, then check the situation again.(can be executed when for ex. Delivered all available letters and waits for new once to appear)	1 min

The simulation ends once every letter in the generated run has been delivered. It can also stop at the configured action(steps) limit.

5 Evaluation of possible actions

DeliveryAgent manages the decisions. Its planning component, LookaheadStrategy, estimates what would happen if the courier visited pickup points and delivery addresses in different orders. These estimates use only letters already announced and the roads the agent currently believes are usable.

1. Update remembered road closures using the latest view.
2. If a carried letter belongs at the current address, deliver it. Otherwise, collect a waiting local letter if there is space, starting with the oldest one and then the lowest letter ID.
3. Keep travelling toward the selected stop if the relevant information is unchanged and a route still exists. Reconsider the plan when letters, bag space or remembered closures change.
4. When making a new plan, try alternative orders of pickup and delivery stops. For each alternative, estimate arrival times, bag contents and missed deadlines.
5. Prefer a complete plan with fewer late letters. If plans tie, prefer less total lateness, then an earlier finish. Move one cell toward the first chosen stop and observe again.
6. If the main planner cannot build a complete plan, use a simpler delivery and pickup check. If no useful action is found, wait for one minute.

Choosing an order and finding a road

The order of visits and the road between two visits are separate problems. The planner chooses which stop to visit next. Breadth-first search (BFS), finds a road to that stop by checking cells one move away, then two moves away, and so on. Because every successful move costs the same time, this finds a shortest road on the map of currently known obstacles. For equally short roads, the agent prefers one passing other carried-letter addresses.

Beam search builds routes one stop at a time. At each round, it extends the current partial routes with reachable pickup or delivery stops, then ranks them by the estimated number of late letters, travel time and progress already made. It keeps up to 64 routes with the best scores for the next round. The search runs at worst-case $2N + 1$ rounds, where N is the number of letters included in the plan. Limiting the search saves computation time, but may discard a route that would later have been better.

6 What the agent remembers

The agent keeps its building map, the locations of observed closures, when it last saw them and its selected delivery or pickup target. A closure is deleted from memory after 12 simulated minutes without a new sighting. If the courier sees that cell is open, it removes the closure from memory of blocked cells as well.

These memories are not a perfect picture of the city. A remembered closure may already have ended, while an unseen closure may be there on map. A new run starts with a new agent and empty memory.

7 Agent

The courier does not follow a fixed list of addresses. It uses incoming information, remembers road closures and compares the expected results of different visit orders. This is the sense in which it makes intelligent decisions for its delivery task.

Here, doing the right thing means choosing actions that help deliver as many letters on time as possible. When comparing route plans, the agent first prefers fewer late letters, then less total lateness, and then a shorter completion time.

Its rationality is bounded by missing knowledge of future arrivals and unseen closures, and by the beam search limit of 64 partial plans. A reasonable route may later be blocked by a newly observed closure. The agent therefore cannot guarantee an optimal route or an on-time delivery for every letter.

The agent uses earlier observations within the current run through its closure memory, each run starts with fresh memory.

The agent chooses based on its current information, but the actual outcome depends on what happens during the run. Its intelligence comes from planning and adapting to current observations.

8 Results

A deadline is the latest time at which a letter counts as delivered on time. It is calculated when the courier collects the letter: pickup time plus the allowed delivery time. For example, a letter collected at minute 30 with an allowance of 120 minutes has a deadline of 150. Delivery at minute 150 is on time; delivery at 151 is late.

The on-time rate is the number delivered on time divided by all letters in the run, including any still undelivered or not yet available. For example, 8 on-time deliveries out of 10 letters gives 80%. The program also records how many letters were delivered, their total lateness, the number of actions and the journey duration.

9 Experiments

Was tested three configurations with 20 runs each. Every run used three pickup points, a bag capacity of 10 letters and a limit of 2000 actions. Temporary closures lasted 12–36 minutes, and closure memory lasted 12 minutes. Delivery allowances started when each letter was picked up.

Setting	10 × 10	15 × 15	20 × 20
Letters per run	10	15	20
Buildings	10	22	40
Temporary closures	4	6	8
Letter arrival window (min)	200	500	700
Delivery allowance (min)	120	160	200
Closure start window (min)	180	420	580
Experiment	004	005	006

The arrival windows were adjusted with reference to the average simulated run times observed in the earlier results. Longer windows spread new arrivals across the longer runs on larger maps. The seed is set, but we can change it any time, and the program can generate repeatable runs from duplicate seeds.

These scenarios show how the agent performs as the complexity of the task grows. Because several values changed together.

10 Results and interpretation

The main measure is letters delivered on time per run, shown as X/N. The table reports the mean and sample standard deviation (SD) across 20 runs. SD describes variation between scenarios. All 60 runs finished within the action limit.

Map	On time mean / total	SD letters	Simulation time mean $\pm$ SD min	Actions mean $\pm$ SD
10 $\times$ 10	9.55 / 10	0.69	418.9 $\pm$ 60.6	91.2 $\pm$ 10.2
15 $\times$ 15	12.75 / 15	1.37	936.8 $\pm$ 94.8	188.6 $\pm$ 15.8
20 $\times$ 20	16.40 / 20	0.88	1465.5 $\pm$ 142.4	284.3 $\pm$ 23.7

These counts correspond to 95.5%, 85.0% and 82.0% on-time delivery. The larger configurations required more actions and had a lower on-time share. Their longer arrival windows also affect total simulation time, so duration alone does not measure route quality. Mean recorded execution times were 0.079, 0.418 and 1.387 seconds per run on the machine used.

Each figure below keeps one configuration separate. Bars show on-time deliveries; the lower line shows simulated completion time. Run numbers identify independent scenarios, not learning progress. The time axes use different scales.

10 by 10 map

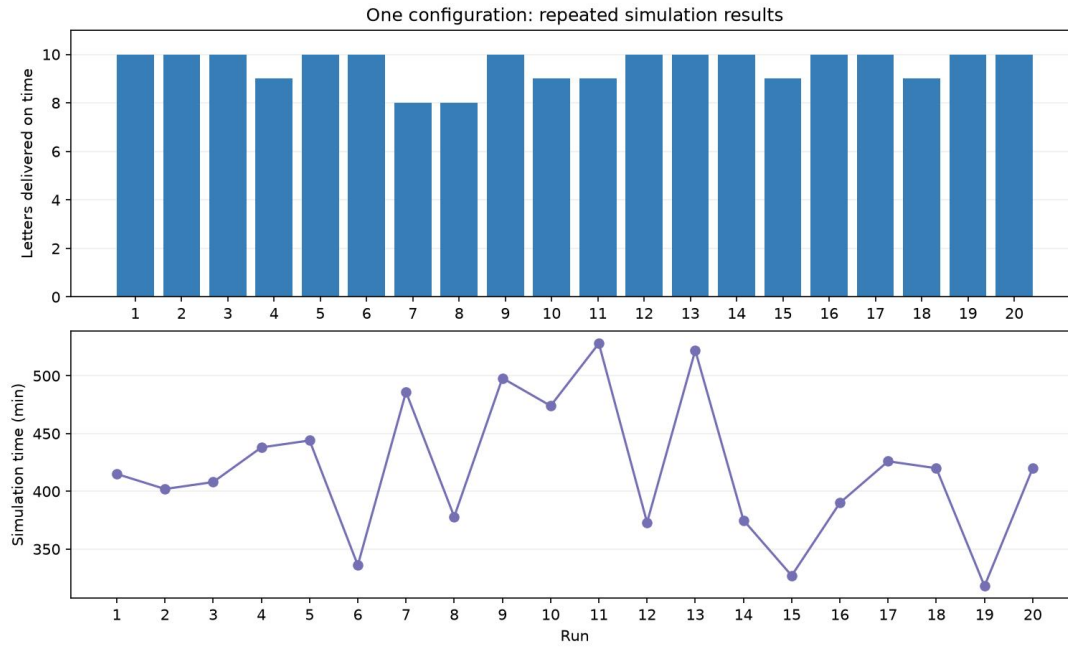


Figure 1. 10×10 map, 20 independent runs, experiment_004.

The agent delivered 8–10 letters on time, with all 10 on time in 13 runs. Run 13 took 522 minutes with 10/10 on time, while run 8 took 378 minutes with 8/10. A shorter overall run therefore did not necessarily meet more deadlines.

15 by 15 map

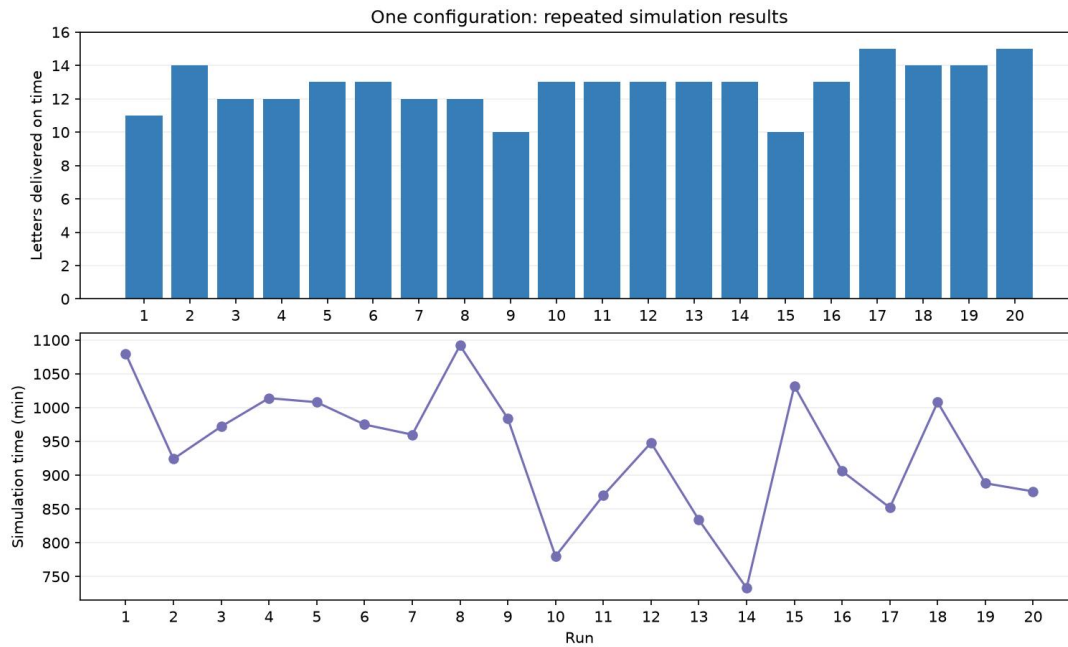


Figure 2. 15×15 map, 20 independent runs, experiment_005.

The results ranged from 10/15 to 15/15 on time; two runs met every deadline. This configuration had the largest variation in on-time share (SD 9.14 percentage points), indicating greater sensitivity to the generated scenario.

20 by 20 map

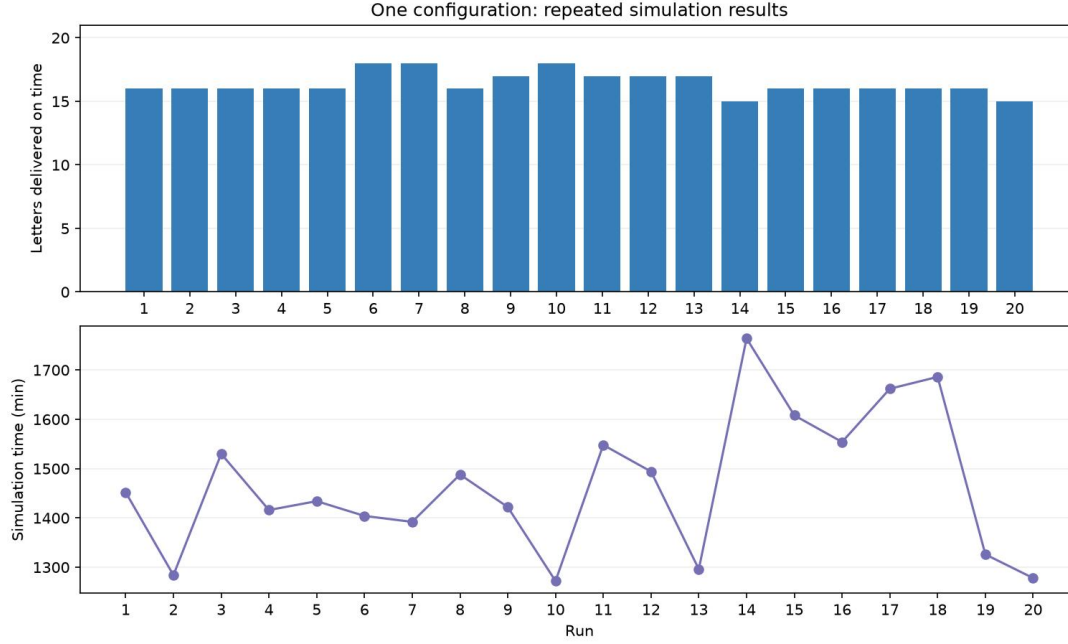


Figure 3. 20×20 map, 20 independent runs, experiment_006.

The agent delivered 15–18 of 20 letters on time. No run met every deadline. The on-time share varied less than on the 15×15 map (SD 4.41 percentage points), but every run still had at least two late deliveries.

A decision traced through the history and code

In experiment_004, run_018, letter 2 was collected at minute 66, due at 186 and delivered at 282: 96 minutes late. Replaying the saved scenario with the available source code reproduced all 90 recorded actions. The following events show how other deliveries came first.

Time (min)	Recorded event
66	Collect letters 2, 7, 3 and 9 at (1, 2).
144	Collect letters 6, 5 and 8 at (7, 5).
162–174	Deliver letters 3, 7 and 6.
192–246	Deliver letters 8 and 5.
282	Deliver letter 2, 96 minutes late.

At minute 174, after delivering letter 6, the shortest known route to letter 2 still needed 36 minutes, but only 12 remained before its deadline. The planner chose letter 8, 18 minutes away and due at 264. LookaheadStrategy ranks complete plans by fewer late letters, then less total lateness, then earlier completion. An earlier deadline therefore does not always take priority. The earlier route may still have been suboptimal.

Waiting before pickup is excluded from deadlines: letter 4 waited 209 minutes and still counted as on time. Mean waits across all letters were 87.3, 206.4 and 341.8 minutes for the three configurations. Comparing this planner with a basic agent remains necessary to establish its benefit.

11 User manual

Open the Delivery_agent project in IDE. Run terminal commands from the project folder containing main.py, requirements.txt, configs and tests.

Install the required packages with pip

The requirements.txt file lists the required packages, including matplotlib and pytest. Install them with:

```
python -m pip install -r requirements.txt
```

Run a configuration

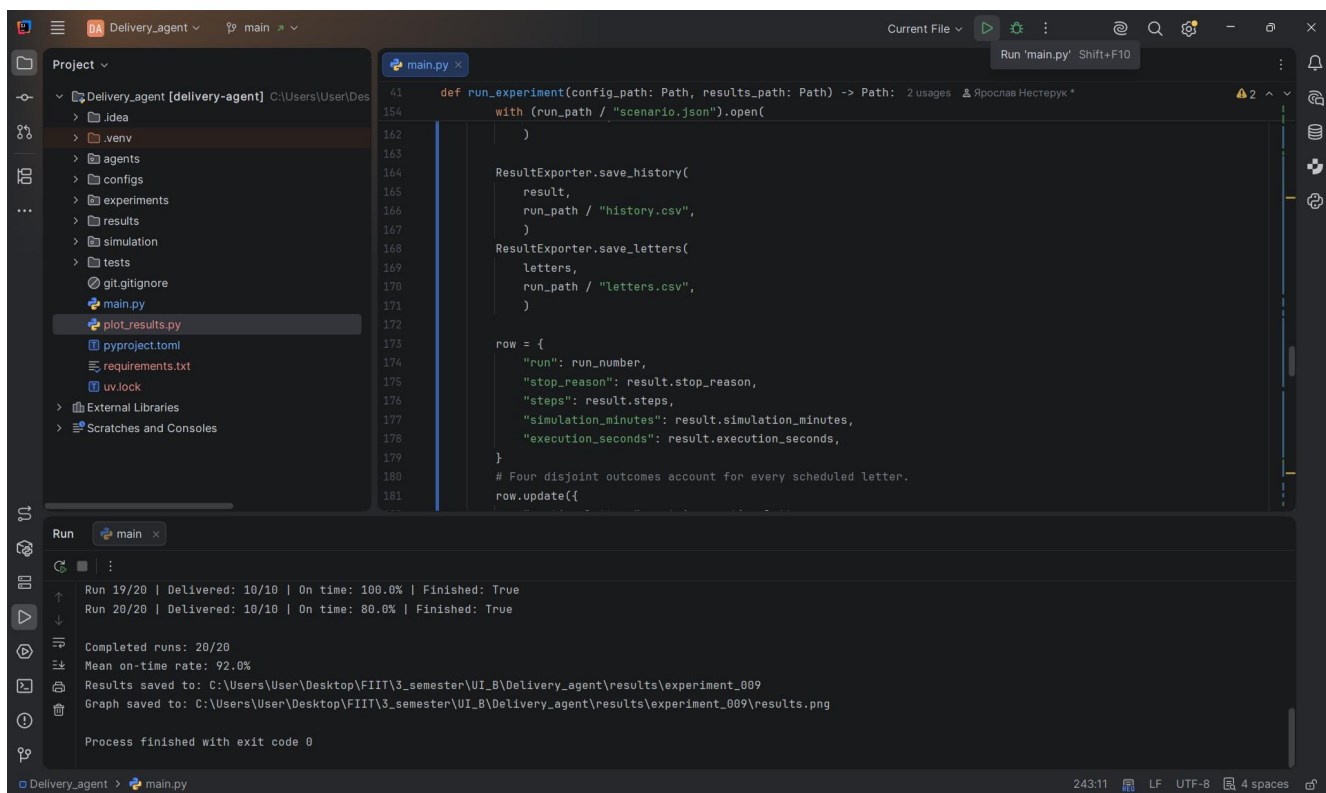
```
python main.py --config configs/map_10.json
python main.py --config configs/map_15.json
python main.py --config configs/map_20.json
```

Run one command at a time. Each supplied experiment preset runs 20 scenarios. The program prints the results and creates a new results/experiment_NNN folder.

Run the default configuration with the button

Open main.py in IDE and click the green Run triangle next to the main entry point, or right-click main.py and select Run. Leave script parameters empty. The program then uses configs/default.json. The equivalent terminal command is:

```
python main.py
```



Find results and change settings

Open results.png for the graphs. runs.csv contains each run's results, aggregate.csv contains means and SD, and settings.json records the experiment settings. Each run folder contains scenario.json, history.csv and letters.csv.

To change a scenario, edit a configuration file: run_count controls repetitions; config controls map size, capacity, delivery allowance and seed; generation controls letters, buildings and closures. For a short demonstration, set run_count to 1. Pass the file to --config as shown above.

Run all tests and show each result

```
python -m pytest tests -v
```

Run this command from the project folder. If a required module is missing, repeat the package installation command using the same project interpreter.

12 Libraries and external sources

IN project was used Python standard library, files, random generation and seed hashing. Tests use unittest classes and can be run with pytest. The requirements.txt file contains matplotlib $\geq 3.10, < 4$ for graphs and pytest for running tests.

Sources:

- Russell & Norvig — *Artificial Intelligence: A Modern Approach* — agent theory, PEAS, rationality, partial observability. [Book — Artificial Intelligence: A Modern Approach](#)
- CP-Algorithms — *Breadth First Search* — BFS and shortest paths in unweighted graphs. [Breadth First Search](#)
- Meister et al. — *Best-First Beam Search* — bounded Beam Search. [Best-First Beam Search](#)
- Python docs — collections.deque — BFS queue implementation. [Python collections documentation](#)
- Python docs — abc and random — strategy abstraction and reproducible random experiments. [Python abc documentation](#) [Python random documentation](#)
- Chat GPT - helped with plotting results and test